

Présentation : de la configuration par défaut à la configuration optimisée

Dans les modules précédents, vous avez utilisé les paramètres de modèle par défaut :

```
# Dans les modules 1-2
agent = LlmAgent(
    model="gemini-2.5-flash",
    instruction="..."
)
```

Cette approche fonctionne, mais utilise les paramètres par défaut pour toutes les opérations :

-  Aucun contrôle sur la créativité par rapport à la cohérence
-  Aucun seuil de sécurité configuré
-  Aucune limite de jeton définie
-  Mêmes paramètres utilisés pour toutes les tâches (écriture créative et extraction de données)

Dans ce module, nous allons apprendre à configurer les modèles selon des exigences spécifiques à l'aide de `generate_content_config` d'ADK.

Le problème : une solution unique ne convient pas à tous les cas

L'utilisation des paramètres par défaut ou d'un modèle inadapté entraîne un gaspillage de ressources ou une baisse de qualité :

```
# Problème 1 : modèle coûteux pour des tâches simples
greeting_agent = LlmAgent(
    model="gemini-2.5-pro", # Sursollicitation ! Coûte 10 fois plus cher que Flash
    instruction="Greet the user warmly"
)
```

```
# Problème 2 : température inadapté pour la tâche
factual_agent = LlmAgent(
    model="gemini-2.5-flash",
    instruction="Extrais les dates exactes des documents"
    # Aucune température configurée - 1.0 par défaut (créativité/randomisation élevées)
    # Pour les tâches factuelles, la température doit être proche de 0 !
)
```

```
# Problème 3 : aucune configuration de sécurité personnalisée
public_agent = LlmAgent(
    model="gemini-2.5-flash",
    instruction="Réponds aux questions des clients"
    # Utilise les paramètres de sécurité par défaut - peut ne pas répondre aux
```

exigences de conformité spécifiques
)

Le problème principal : sans configuration stratégique, les agents gaspillent de l'argent, produisent des résultats incohérents ou ne respectent pas les exigences spécifiques liées à la sécurité et à la conformité.